CLAUDE CODE · INVENTARIO LOCAL

Manual de skills instaladas

Estado de `~/.claude/skills/` y `~/.claude/plugins/` en este equipo con Windows: ui-ux-pro-max y claude-mem documentados a fondo, cuatro skills de referencia que aún no están instaladas, y el resto del catálogo en un índice.

63	7	1	56
SKILLS TOTALES	UI-UX-PRO-MAX	PLUGIN (CLAUDE-MEM)	SUITE GSTACK + OTRAS

`security-audit` `ui-ux-pro-max` `claude-mem` `brag` `diagram-design` `anatomy` `rea` `markitdown` `gstack` `Referencia: ECC` `Gestión de skills`

- **Estado verificado el 17 oct 2026 en este PC con Windows:** hay 63 skills en `~/.claude/skills/` (las 7 de ui-ux-pro-max y la suite `gstack` v1.60.1.0) y el plugin `claude-mem` v13.28.0. `security-audit`, `brag`, `diagram-design` y `anatomy` están descritas abajo como referencia, pero **no están instaladas** en este equipo.

security-audit

Destino previsto: ~/.claude/skills/security-audit/

NO INSTALADA · 1 SKILL · CLOUDFLARE

QUÉ HACE

Convierte al agente en auditor de seguridad. Orquesta agentes aislados a lo largo de seis fases: reconocimiento, caza guiada por cobertura, validación de candidatos, salida estructurada (`findings.json`), verificación independiente de los registros y reporte neutral (`REPORT.md`, `FINDINGS-DETAIL.md`, `NEEDS-VALIDATION.md`). Es la skill que dio origen al arnés de descubrimiento de vulnerabilidades de Cloudflare.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# clonado desde
git clone https://github.com/cloudflare/security-audit-skill

# copiado skills/security-audit/ a
~/.claude/skills/security-audit/

# alternativa oficial (Skills CLI) no usada aquí
npx skills add https://github.com/cloudflare/security-audit-skill \
  --skill security-audit --global
```

↗ Repositorio en GitHub ↗ Blog de Cloudflare

CÓMO SE USA

- Pedidos como `security audit this codebase`, `find security vulnerabilities in ./src` o `do a security review, output to ~/audits/mi-proyecto`.
- **Modo guía** (por defecto): preguntas de seguridad o revisiones puntuales, sin crear archivos.
- **Modo auditoría completa**: solo si pides auditar o hacer pen-test de un repo, o reportes. Salida por defecto en `~/security-audit-skill/<repo>/run-N`.
- Varias ejecuciones sobre el mismo repo son aditivas: una sola corrida encontró aprox. la mitad de lo que encontraron las repetidas.

REQUISITOS

Node.js para los validadores (`validate-findings.cjs`, `validate-coverage-ledger.cjs`; sin dependencias). Aquí hay Node `v24.18.0`. Para ejecutar código del objetivo exige un sandbox del SO sin red; sin él, deja el hallazgo como `needs_validation` en vez de ejecutar nada.

ui-ux-pro-max

`~/.claude/skills/{ui-ux-pro-max, design, design-system, ui-styling, brand, banner-design, slides}/`

7 SKILLS

QUÉ HACE

Inteligencia de diseño UI/UX searchable en local: 79 estilos visuales (50 activos), 192 paletas de color con perfil de razonamiento por tipo de producto, 74 combinaciones tipográficas, 119

guías de UX, 105 iconos curados, 17 presets de GSAP, 25 tipos de gráfico y 22 stacks técnicos (React, Next.js, Vue, Nuxt, Svelte, Astro, SwiftUI, Flutter, Tailwind, shadcn/ui, Jetpack Compose, Angular, Laravel, Three.js, y más).

El plugin trae siete skills relacionadas, no solo una. El repo publica el paquete completo bajo `.claude/skills/`, así que instalar "el skill" instala las siete.

INSTALACIÓN (YA APLICADA)

```
# clonado desde
git clone https://github.com/nextlevelbuilder/ui-ux-pro-max-skill.git

# copiados los 7 directorios de .claude/skills/ del repo a
~/.claude/skills/ui-ux-pro-max/
~/.claude/skills/design/
~/.claude/skills/design-system/
~/.claude/skills/ui-styling/
~/.claude/skills/brand/
~/.claude/skills/banner-design/
~/.claude/skills/slides/

# alternativas oficiales no usadas aquí
/plugin marketplace add nextlevelbuilder/ui-ux-pro-max-skill
/plugin install ui-ux-pro-max@ui-ux-pro-max-skill

npx ui-ux-pro-max-cli init --ai claude --global
```

↗ Repositorio en GitHub ↗ uupm.cc

- Se copiaron los archivos estáticos directo del repo en vez de correr `npx ui-ux-pro-max-cli` — evita ejecutar el instalador npm de un tercero; el resultado en disco es el mismo.

LAS 7 SKILLS DEL PAQUETE

ui-ux-pro-max
Motor principal: búsqueda de estilos, paletas, tipografía, charts y reglas UX/accesibilidad por prioridad.

design
Identidad de marca, tokens, logos (55 estilos), CIP, banners, iconos, fotos sociales.

design-system
Arquitectura de tokens en tres capas (primitivo→semántico→componente) y specs de componentes.

ui-styling
Componentes accesibles con shadcn/ui (Radix +

brand
Voz de marca, mensajería, gestión de

banner-design
Banners para redes, ads y hero de sitio — 22 estilos, múltiples

Tailwind) y diseños basados en canvas.

assets y consistencia de guías de estilo.

formatos.

slides

Presentaciones HTML con Chart.js, tokens de diseño y fórmulas de copywriting.

CÓMO SE DISPARA

- → Automático al diseñar, construir o revisar cualquier UI: páginas, componentes, paletas, tipografía, layout, accesibilidad, animación.
- → También responde a pedidos explícitos de logo, banner, CIP, icono o presentación HTML.
- → Se salta en lógica de backend, APIs, infraestructura o scripts sin impacto visual.

REQUISITO

Python 3.x para el script de búsqueda (solo librería estándar, no instala nada ni hace llamadas de red). Ya está presente en esta máquina: `Python 3.14.6`.

claude-mem

`~/.claude/plugins/marketplaces/thedotmack/ · cache en .../cache/thedotmack/claude-mem/13.28.0`

PLUGIN + WORKER

QUÉ HACE

Sistema de memoria persistente para Claude Code. Captura observaciones de uso de herramientas durante la sesión, genera resúmenes semánticos y los deja disponibles para sesiones futuras del mismo proyecto — sin que el usuario tenga que repetir contexto.

A diferencia de ui-ux-pro-max, no es un conjunto de archivos estáticos: registra hooks de ciclo de vida en Claude Code y corre un worker HTTP en segundo plano.

INSTALACIÓN Y CONFIGURACIÓN (INSTALADO EL 27 SEP 2026)

```
# instalado sin cuenta / sin nube, con el plan de Anthropic ya logueado
CLAUDE_MEM_ONLINE_OPTIN=false npx claude-mem@latest install \
  --ide claude-code --provider claude
```

```
# telemetría anónima apagada
npx claude-mem telemetry disable

# worker local iniciado manualmente (el installer lo dejó sin arrancar)
npx claude-mem start
```

↗ Repositorio en GitHub ↗ Web viewer del worker

ESTADO ACTUAL

CONFIG	VALOR
Proveedor	`claude` — usa la sesión de Anthropic ya logueada, sin API key aparte
Cloud sync	OFF — nada se manda a cmem.ai salvo el contacto único de alta si te logueas
Telemetría	OFF — deshabilitada explícitamente tras instalar
Worker	Corriendo en `http://127.0.0.1:37777`
Datos	Todo local en `~/.claude-mem/`

- El worker responde en `http://127.0.0.1:37777` (estado *ok*, v13.28.0). La memoria se acumula al usar Claude Code en un proyecto y se inyecta desde la segunda sesión de ese proyecto.

SKILL PRINCIPAL + COMANDOS

El plugin expone `mem-search` como la skill de cara al usuario (búsqueda de observaciones pasadas), más ~19 skills internas de soporte (`learn-codebase`, `how-it-works`, `knowledge-agent`, `standup`, `agent-cost-report`, `weekly-digests`, entre otras) para flujos específicos del propio proyecto claude-mem.

COMANDO	QUÉ HACE
`npx claude-mem status`	Estado del worker (PID, puerto, uptime, dependencias)
`npx claude-mem doctor`	Diagnóstico de runtime (Bun, uv, worker, plugin)
`npx claude-mem start / stop / restart`	Control manual del worker en segundo plano

`npx claude-mem search "<query>"`	Busca observaciones guardadas desde la terminal
`npx claude-mem telemetry status`	Confirma que la telemetría sigue apagada
`npx claude-mem uninstall`	Quita el plugin, hooks y configuración

brag

NO INSTALADA · 1 SKILL · VIDEO

Destino previsto: ~/.claude/skills/brag/

QUÉ HACE

Convierte el proyecto actual en un video corto de lanzamiento, con música, movimiento y copy para compartir. Lee el código del proyecto directamente, sin necesitar URL en vivo ni capturas de pantalla. La composición del video la ejecuta Hyperframes; la skill decide el ángulo de producto, el tono y qué momentos mostrar.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# clonado desde
git clone https://github.com/latent-spaces/brag.git

# copiado skills/brag/ a
~/.claude/skills/brag/
  ├── SKILL.md
  ├── assets/    # música y efectos de sonido incluidos
  ├── references/
  └── scripts/

# alternativa con el instalador oficial (cualquier agente)
npx skills add https://github.com/latent-spaces/brag --skill brag

# o como plugin nativo de Claude Code
/plugin marketplace add latent-spaces/brag
/plugin install brag@brag
```

↗ Repositorio en GitHub ↗ Ver en skills.sh ↗ Hyperframes

REQUISITOS PARA EJECUTARLA

→ Node.js 22+

- → FFmpeg disponible en `PATH`
- → CLI de Hyperframes; verifícalo con `npx hyperframes doctor`

USO BÁSICO

1. Párate en la carpeta del proyecto que quieres mostrar.
2. Escribe `let's /brag` o simplemente `/brag`.
3. La skill lee el código y arma un guion y un storyboard cortos.
4. Entrega un brief de composición a Hyperframes, que renderiza el video.
5. Recibes una carpeta `brag-output/` con plan, brief, copy para redes y `brag.mp4`.

OPCIONES (FLAGS)

FLAG	VALORES	POR DEFECTO
`--tone`	preset o descripción libre	inferido
`--format`	`landscape` / `vertical` / `square`	`landscape`
`--duration`	segundos	auto (15–25 s)
`--no-music`	flag	música activada
`--no-sfx`	flag	efectos activados
`--title`	texto	inferido del proyecto
`--voice`	flag	narración apagada

Los presets de tono son `default`, `polished`, `yc-parody`, `chaotic`, `deadpan`, `cinematic` y `app-store`. También acepta dirección libre:

```
/brag --tone "fake Series A launch from 2016"
/brag --voice
```

- La narración usa Kokoro vía Hyperframes y solo se activa con `--voice`. Los requisitos (FFmpeg, Hyperframes) no se han verificado en esta máquina.

diagram-design

Destino previsto: ~/.claude/skills/diagram-design/

NO INSTALADA · 1 SKILL · 45 TIPOS

QUÉ HACE

Dibuja diagramas como un único archivo HTML con SVG incrustado, con estilo editorial y sin sombras. Cubre 45 tipos: arquitectura, flujo, secuencia, máquina de estados, ER, línea de tiempo, swimlane, cuadrante, radar, Sankey, Gantt, mapa de Wardley, kanban, UML, esquema de base de datos y más. El agente elige el tipo, explica su plan y escribe el archivo, que se abre con doble clic.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# clonado desde
git clone https://github.com/cathrynlavery/diagram-design.git

# copiado a
~/.claude/skills/diagram-design/
  ├── SKILL.md
  ├── assets/
  ├── references/
  └── scripts/   # importadores y exportador, solo Python local

# alternativa con el instalador oficial (cualquier agente)
npx skills add cathrynlavery/diagram-design

# o como plugin de Claude Code (añade comandos slash)
/plugin marketplace add cathrynlavery/diagram-design
/plugin install diagram-design@diagram-design
```

↗ Repositorio en GitHub ↗ Ver en skills.sh ↗ Sitio web ↗ Galería

USO BÁSICO

1. Pide el diagrama en lenguaje natural, por ejemplo `Hazme un diagrama de arquitectura: frontend, backend, base de datos y caché Redis`.
2. La primera vez en un proyecto, pregunta si quieres adaptar el estilo a tu marca: URL, skill instalada, carpeta, tokens pegados, estilo por defecto o perfil guardado.
3. El agente indica qué tipo de diagrama va a usar y escribe un archivo `.html`.

4 Abre el archivo en el navegador, o pide exportarlo a PNG o SVG.

IMPORTAR Y EXPORTAR

- → Importa archivos `.drawio`, Mermaid y `.excalidraw` y los redibuja en tu estilo.
- → Exporta a PNG o SVG con `/export-diagram` (versión plugin).
- → `/doctor` revisa que la instalación esté completa (versión plugin).

- Sin servidor, cuentas ni telemetría. Lo único que sale a internet son las fuentes de Google Fonts cuando abres un diagrama en el navegador.

anatomy

NO INSTALADA · 1 SKILL · FIGURAS

Destino previsto: ~/.claude/skills/anatomy/

QUÉ HACE

Explica una idea técnica construyéndola como un objeto físico que funciona. Una clave de caché que evita trabajo repetido se vuelve una cerradura de pines que solo gira si todos los pines coinciden. Genera figuras isométricas interactivas en SVG, con muchas piezas pequeñas sombreadas, dentro de una tarjeta con título, pista y lectura en vivo. Puede añadir shaders WebGL (fuego, agua, luz) dentro del mismo dibujo.

Cada número que muestra la figura sale del mismo modelo que mueve sus piezas. Funciona sin framework (TypeScript/JS que escribe SVG y HTML) y en React/Next.js.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# el repositorio completo es la carpeta de la skill
git clone https://github.com/wheresryan22/anatomy.git
# copiado (sin .git) a
~/.claude/skills/anatomy/

# alternativa con el instalador oficial
npx skills add wheresryan22/anatomy -g -a claude-code

# otros agentes: cambia -a por opencode o codex
```

↗ Repositorio en GitHub ↗ Sitio web ↗ Descargar ZIP

REQUISITOS

- Node.js 20.10 o superior. Aquí hay Node `v24.18.0`.
- Chrome o Chromium para las comprobaciones de la figura. No verificado en esta máquina.
- Sin API keys ni `npm install`: usa solo módulos integrados de Node.

USO

```
/anatomy explain how a token bucket rate limiter works, interactive
```

- También se activa solo ante pedidos de ilustraciones isométricas, figuras explicativas para docs o blogs, o SVG que muestren cómo funciona un sistema.
- El repo incluye 8 ejemplos en `examples/` (motor Raptor, tanque de ondas, indicador de dial, máquina arcade y otros).
- Puede publicar una figura terminada en la galería comunitaria, solo si se lo pides.

- Los scripts de captura solo lanzan Chrome local y se comunican con `127.0.0.1`. No hay llamadas a servicios externos en el código de la skill.

rea · Reverse Engineer Anything

Destino previsto: MCP + skill `reverse-engineer-anything` · paquete npm `rea-agents` v6.5.0

NO INSTALADA · MCP + 1 SKILL

QUÉ HACE

Un solo servidor MCP para hacer ingeniería inversa de binarios, aplicaciones y comportamiento en ejecución. Ves una función en una app, le pides al agente que investigue cómo funciona con REA y te devuelve la explicación con la evidencia y sus límites, sin necesitar el código fuente. El análisis corre en local y también se puede usar desde la terminal.

Cubre binarios nativos (con Hopper, Ghidra o IDA), JavaScript/Electron, ensamblados .NET, APK de Android, sitios web, firmware, capturas de red y de paquetes, bases SQLite, WASM y bytecode EVM, entre otros.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# registra el MCP y las instrucciones de flujo en tu agente
npx rea-agents setup

# CLI para uso normal
npm install --global rea-agents
rea --help

# ejemplo desde la terminal: app JavaScript/Electron extraída o ASAR
npx -y rea-agents@latest analyze-javascript-application /ruta/absoluta/app --
```

Setup muestra los cambios, pide aprobación, guarda copias de la configuración existente y soporta Claude Code, Codex, Cursor, Gemini CLI y otros agentes. Reinicia el agente después.

↗ Repositorio en GitHub ↗ rea.tools ↗ Ver en skills.sh ↗ npm

CÓMO SE USA

```
Understand how search works in the Notes app, show me the evidence,
and build a similar feature for my project.
```

- La skill `reverse-engineer-anything` se activa cuando una conclusión depende de un binario o paquete ya distribuido, de decompilación, de evidencia pasiva en ejecución o de comparar versiones. Se salta el análisis de arquitectura de un repo con código fuente.
- El repo trae además 4 skills de mantenimiento propias del proyecto en `.agents/skills/` (`rea-tool-design`, `rea-analysis-optimization`, `rea-ci-tests-audit`, `rea-changelog-update`); no hacen falta para usar REA.

REQUISITOS

- Node.js 22.x (≥22.19), 24.x (≥24.11) o 26+, más npm.
- Binarios nativos: Hopper, Ghidra o IDA ya instalados. Android: JADX y un JDK completo. Sitios web: un navegador de la familia Chrome.
- En Linux, algunos análisis piden un `ps` compatible con procps en el `PATH`.

- Licencia MIT; el repo incluye un `DISCLAIMER.md` y `SECURITY.md`. Descárgalo solo desde github.com/morluto/rea, rea.tools o el paquete npm `rea-agents`: el propio README advierte que un nombre parecido no garantiza que sea oficial. La captura en ejecución usa tus permisos de usuario, así que revisa la guía de cada modo antes de analizar software ajeno. Datos tomados del README; no instalé ni ejecuté REA.

markitdown

Paquete PyPI `markitdown` v0.1.8 · repo microsoft/markitdown

NO INSTALADA · CLI + MCP

QUÉ HACE

Utilidad ligera de Python (de Microsoft) que convierte archivos a Markdown para usarlos con LLMs y flujos de análisis de texto. Conserva la estructura importante (títulos, listas, tablas, enlaces). La salida está pensada para que la lean herramientas de análisis, así que no es la mejor opción para conversiones de alta fidelidad destinadas a personas.

No es una skill ni un `SKILL.md` : es una herramienta de línea de comandos y una librería. Se incluye aquí porque se puede conectar a un agente como servidor MCP.

Formatos: PDF, PowerPoint, Word, Excel, imágenes (EXIF y OCR), audio (EXIF y transcripción), HTML, CSV/JSON/XML, ZIP (recorre el contenido), URLs de YouTube, EPub y más.

INSTALACIÓN (PENDIENTE EN ESTE EQUIPO)

```
# Python 3.10 a 3.14; conviene usar un entorno virtual
python -m venv .venv
source .venv/bin/activate
pip install 'markitdown[all]'

# solo algunos formatos
pip install 'markitdown[pdf, docx, pptx]'
```

↗ Repositorio en GitHub ↗ PyPI ↗ markitdown-mcp

USO

```
markitdown archivo.pdf > documento.md
markitdown archivo.pdf -o documento.md
cat archivo.pdf | markitdown
```

OPCIÓN	QUÉ HACE
[all]	Instala todas las dependencias opcionales.

`[pdf]` `[docx]` `[pptx]` `[xlsx]` `[xls]`	Dependencias por formato de documento.
`[outlook]`	Mensajes de Outlook.
`[audio-transcription]`	Transcripción de audio wav y mp3.
`[youtube-transcription]`	Transcripción de videos de YouTube.
`[az-doc-intel]` `[az-content-understanding]`	Conversión en la nube con Azure; son llamadas de pago.
`--use-plugins` · `--list-plugins`	Activa o lista plugins de terceros (desactivados por defecto).

PAQUETES DEL REPO

`markitdown`
Librería y CLI principal.

`markitdown-mcp`
Servidor MCP (STDIO, HTTP y SSE) con una herramienta: `convert_to_markdown(uri)` .

`markitdown-ocr`
Plugin de OCR con visión de LLM para PDF, DOCX, PPTX y XLSX.

COMO SERVIDOR MCP

```
pip install markitdown-mcp
markitdown-mcp                                  # STDIO (por defecto)
markitdown-mcp --http --host 127.0.0.1 --port 3001
```

Acepta URIs `http:` , `https:` , `file:` y `data:` . Para Claude Desktop el README recomienda la imagen de Docker; para leer archivos locales hay que montar el directorio, por ejemplo `-v /home/user/data:/workdir` .

- **Seguridad:** MarkItDown lee archivos y URLs con los privilegios del proceso que lo ejecuta, como `open()` o `requests.get()` . Con entradas no confiables, usa la función `convert_*` más estrecha posible. El servidor MCP es solo para uso local: escucha en `localhost` por defecto, y no conviene enlazarlo a otras interfaces. Datos tomados del README; no instalé ni ejecuté la herramienta.

gstack

SUITE · 57 SKILLS

~/.claude/skills/gstack/ · repo garrytan/gstack · v1.91.126

QUÉ HACE

Convierte a Claude Code en un equipo de ingeniería virtual: un CEO que repiensa el producto, un eng manager que fija la arquitectura, un diseñador que caza el "AI slop", un revisor que encuentra bugs de producción, un QA que abre un navegador real, un oficial de seguridad y un release engineer que envía el PR. Son comandos slash en Markdown, con licencia MIT, creados por Garry Tan (presidente y CEO de Y Combinator).

No es una colección de herramientas sueltas sino un proceso, y las skills corren en el orden de un sprint, cada una alimentando a la siguiente:

```
Think → Plan → Build → Review → Test → Ship → Reflect
```

INSTALACIÓN

```
# desde Claude Code, o en terminal
git clone --single-branch --depth 1 https://github.com/garrytan/gstack.git \
  ~/.claude/skills/gstack
cd ~/.claude/skills/gstack && ./setup

# modo equipo (desde dentro de tu repo): auto-actualización para todos
(cd ~/.claude/skills/gstack && ./setup --team) \
  && ~/.claude/skills/gstack/bin/gstack-team-init required
```

↗ Repositorio en GitHub

REQUISITOS

- Claude Code y Git.
- Bun 1.4.2+ (versión probada; `./setup` rechaza Bun anterior a 1.3.3). Node.js solo en Windows.
- Recomendado en macOS: el navegador Aside; sin él, `./setup` construye el navegador propio de gstack.
- `/cso` necesita además un Bun con las cuatro opciones `--no-compile-autoload-*` y un compilador nativo; si faltan, reporta *not assessed* y el resto sigue funcionando.

PRIMEROS PASOS

1. `/office-hours`: describe lo que estás construyendo.
2. `/plan-ceo-review` sobre cualquier idea de feature.
3. `/review` sobre cualquier rama con cambios.
4. `/qa` sobre tu URL de staging o una API local aislada.

QUÉ REVISIÓN USAR

CONSTRUYES PARA…	ANTES DEL CÓDIGO	AUDITORÍA EN VIVO
Usuarios finales (UI, web, móvil)	`/plan-design-review`	`/design-review`
Desarrolladores (API, CLI, SDK, docs)	`/plan-devex-review`	`/devex-review`
Arquitectura (datos, rendimiento, tests)	`/plan-eng-review`	`/review`
Todo lo anterior	`/autoplan`	—

PENSAR Y PLANEAR

SKILL	QUÉ HACE
`/office-hours`	YC Office Hours: seis preguntas que replantean el producto antes de escribir código. Su documento de diseño alimenta a las demás skills.
`/plan-ceo-review`	Modo CEO/fundador: repiensa el problema. Cuatro modos: Expansion, Selective Expansion, Hold Scope y Reduction.
`/plan-eng-review`	Modo eng manager: fija arquitectura, flujo de datos, casos límite y tests.
`/plan-design-review`	Diseñador senior: puntúa cada dimensión de 0 a 10, detecta "AI slop" y edita el plan.
`/plan-devex-review`	Revisión interactiva de developer experience: personas, TTHW frente a la competencia y puntos de fricción.
`/design-consultation`	Construye un sistema de diseño completo y escribe `DESIGN.md`.
`/autoplan`	Corre CEO → diseño → DX → ingeniería en secuencia y solo te consulta las decisiones de gusto.

`/spec`	Convierte una intención vaga en un spec ejecutable en cinco fases.
`/eval-plan`	Preregistra una evaluación de pago antes de la primera llamada: barras, brazos, reglas de parada y piloto de tres ítems.

CONSTRUIR Y NAVEGAR

SKILL	QUÉ HACE
`/design-shotgun`	Genera 4–6 variantes de mockup, abre un tablero de comparación y recoge tu feedback.
`/design-html`	Convierte un mockup en HTML de producción; detecta React, Svelte o Vue.
`/browse`	Navegador para el agente: usa Aside si está, si no el Chromium propio de gstack.
`/scrape`	Extrae datos estructurados de una página; `/skillify` lo convierte en una browser-skill permanente.
`/skillify`	Convierte el último `/scrape` exitoso en una skill de navegador.
`/open-gstack-browser`	Lanza el navegador de gstack con sidebar, stealth anti-bot y enrutamiento automático de modelo.
`/setup-browser-cookies`	Copia cookies seleccionadas de tu navegador al navegador de gstack (solo en la vía de respaldo).
`/connect-chrome`	Conecta el navegador de gstack con tu Chrome.
`/pair-agent`	Comparte el navegador con otro agente de IA (OpenClaw, Codex, Cursor...) con tokens acotados y aislamiento por pestaña.

REVISAR Y PROBAR

SKILL	QUÉ HACE
`/review`	Staff engineer: busca los bugs que pasan CI y fallan en producción; autocorrige lo obvio.

`/investigate`	Debugging de causa raíz: sin arreglos sin investigación; se detiene tras 3 intentos fallidos.
`/design-review`	Misma auditoría que el plan, pero corrige: commits atómicos y capturas antes/después.
`/devex-review`	Auditoría en vivo de developer experience; prueba el onboarding de verdad.
`/cso`	Chief Security Officer: auditoría con modelo de la aplicación, hallazgos sustentados y cobertura explícita.
`/test-audit`	Barre los tests que cuestan más de lo que protegen; solo reporta salvo que apruebes un lote.
`/deslop-shared-libs`	Busca extracciones de código compartido que valgan la pena; solo recomendaciones.
`/health`	Dashboard de calidad de código.
`/qa`	QA Lead: explora navegador, API, CLI, jobs y webhooks; reproduce, escribe regresión, corrige y reverifica.
`/qa-only`	QA de solo reporte, con evidencia reproducible y sin tocar código.
`/codex`	Segunda opinión independiente desde OpenAI Codex CLI (review, challenge, consult).
`/claude-code`	Segunda opinión desde Claude Code; disponible en todo arnés menos Claude Code.

ENVIAR Y REFLEXIONAR

SKILL	QUÉ HACE
`/ship`	Release engineer: sincroniza main, corre tests, audita cobertura y docs, verifica, hace push y abre el PR.
`/setup-deploy`	Configuración única para `/land-and-deploy`: plataforma, URL de producción y comandos.
`/land-and-deploy`	Mergea el PR, espera CI y deploy, y verifica la salud en producción.
`/landing-report`	Dashboard de solo lectura de la cola de ship por workspace.

`/canary`	Monitoreo post-deploy: errores de consola, regresiones de rendimiento y fallas de página.
`/benchmark`	Base de tiempos de carga, Core Web Vitals y tamaños de recursos; compara antes/después.
`/benchmark-models`	Benchmark cruzado entre modelos para las skills de gstack.
`/document-release`	Audita el comportamiento cambiado contra la documentación en cada ship.
`/document-generate`	Genera documentación faltante con el marco Diátaxis.
`/retro`	Retrospectiva semanal del equipo; `/retro global` abarca todos tus proyectos y herramientas de IA.

SEGURIDAD Y UTILIDADES

SKILL	QUÉ HACE
`/careful`	Avisa antes de comandos destructivos (r -rf, DROP TABLE, force-push).
`/freeze · /unfreeze`	Restringe las ediciones a un directorio · qu la restricción.
`/guard`	`/careful + /freeze` en u solo comando
`/context-save · /context-restore`	Guarda y restaura el contexto de trabajo.
`/learn`	Revisa, busca poda lo que gstack aprend entre sesione

`/make-pdf`	Markdown a PDF de calida de publicació también `--to html` y `--to docx`.
`/diagram`	Descripción e inglés a diagrama editable: fuen mermaid, `.excalidraw` SVG/PNG.
`/setup-gbrain · /sync-gbrain`	Instala gbrain (PGLite o Supabase) y mantiene indexado el código del rep
`/gstack-upgrade`	Actualiza gsta a la última versión.
`/ios-qa · /ios-fix · /ios-design-review · /ios-clean · /ios-sync`	Suite iOS: QA dispositivo rea corrector de bugs, auditorí HIG, limpieza resincronizaci del debug bridge.

- Datos tomados del README y de las carpetas del repo (v1.91.126); no ejecuté `./setup` ni verifiqué el estado de la instalación local. La versión instalada en tu equipo puede diferir. El README sugiere añadir a `CLAUDE.md` que use `/browse` de gstack para toda navegación web en lugar de las herramientas `mcp__claude-in-chrome__*`. La telemetría tiene su propia sección en el README: revísala antes de activarla.

ECC (Everything Claude Code)

SOLO REFERENCIA

Referencia · no instalado · paquete npm `ecc-universal` v2.2.3

QUÉ ES

Un "sistema operativo de agente" para Claude Code (con adaptadores limitados para Codex, Cursor, OpenCode, Gemini y otros). No es una sola skill: es un kit de agentes, skills, hooks y reglas con licencia MIT que impone un ciclo de ingeniería fijo en lugar de improvisar en cada prompt.

```
plan → test → implement → review → verify → remember → improve
```

QUÉ INCLUYE

71 agentes
Planificación, revisión por lenguaje, reparación de builds, seguridad, arquitectura, base de datos, E2E.

306 skills
TDD, investigación, seguridad, docs, frontend, datos, ML, operaciones, patrones de agentes.

97 comandos
Atajos heredados mientras el proyecto migra a un modelo basado en skills.

Hooks y memoria
Se disparan antes y después de cada herramienta: bloquean o avisan, guardan resúmenes de sesión y aprenden patrones ("instincts").

Reglas
Estándares siempre cargados que eliges por lenguaje o proyecto (Python, Go, Rust, React, Swift...).

AgentShield
Escáner de seguridad para prompts, hooks, MCP, permisos y secretos de tus archivos de agente.

INSTALACIÓN (SI DECIDES USARLO)

```
# guiado; requiere Node 18+, Git y Claude Code 2.1+
npx ecc-universal@2.2.3 setup

# o como plugin de Claude Code (elige un solo método, no los apiles)
ecc@ecc
```

↗ Repositorio en GitHub ↗ ecc.tools

- **Antes de instalar:** solapa con `claude-mem` (memoria), `gstack` (revisión, QA y envío) y `security-audit` , así que habría varios sistemas de hooks y memoria compitiendo,

además de más contexto cargado en cada sesión. Los hooks ejecutan código local en cada acción: revisa el perfil de hooks antes de activarlo. Es un proyecto de un solo mantenedor; el núcleo es gratis y hay una versión de pago (ECC Pro, GitHub App para repos privados).

GESTIÓN DE ESTAS SKILLS

- Todas se activan solas cuando el pedido coincide con su descripción, o al invocarlas con `/nombre-skill`.
- ui-ux-pro-max y la suite gstack están instaladas a nivel de usuario (`~/.claude/skills/`): aplican a cualquier proyecto que abras en esta máquina. Para quitar una, borra su carpeta; para actualizarla, vuelve a clonar y sobrescribe.
- claude-mem es distinto: es un **plugin** gestionado por Claude Code (`~/.claude/plugins/`), con un worker en segundo plano. Se administra con `npx claude-mem <comando>` o `/plugin uninstall claude-mem`, no borrando carpetas a mano.
- Este documento refleja el estado al momento de generarlo — si instalas o quitas algo después, pide que se vuelva a actualizar.

Verificado en este PC (Windows) el 17 oct 2026

63 skills · ui-ux-pro-max (7) + gstack y otras (56) + claude-mem (plugin)